

The ISI logo, a stylized red "ISI" monogram.

WORLD STATISTICS CONGRESS 2025 THE HAGUE

IPS 964 New Developments in Seasonal Adjustment

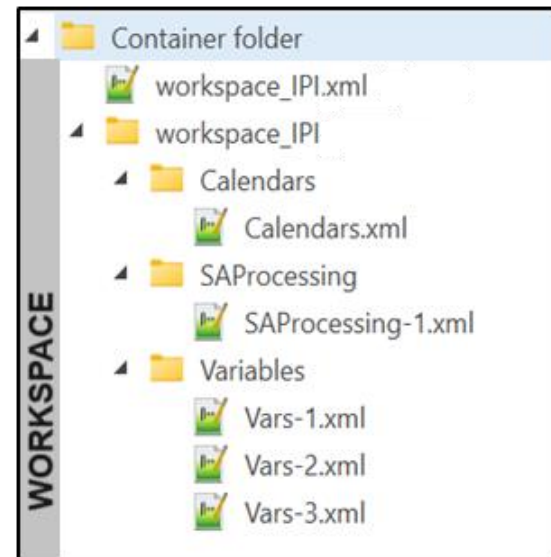
**RJDProcessor: a rjdverse-based library for seasonal
adjustment in official statistics production**

Alessandro Piovani
ISTAT – Italian National Institute of Statistics
Tuesday 7 October 8 a.m. - 9:10 a.m.

Overview



- JDemetra+: recommended software for official statistics production
- Many applications, interfaces and libraries:
 - GUI, Cruncher, **R (rjdverse)**, Java
- **Data + model specifications** stored together in **workspaces (WS)**
 - WS: XML files organized into nested folders
 - Alternative: binary formats (.RData)
- No suggestions are provided on how to organize the storage architecture
 - Options: database, filesystem, GitHub, etc.



Requests and organizational constraints for a seasonal adjustment system



- **Centralized architecture**
 - Web service and/or Web application
 - Automatic application of pre-loaded models when new data are available
 - Easy to be integrated into enterprise and legacy architectures (Java based)
 - Database storage only, no workspaces on the filesystem
- **Model specifications viewable at a glance**
 - No need to use JD+
 - Unfortunately, workspaces are verbose and binary formats are not human readable
- **Processes easy to set up**
 - Orchestration through simple R scripts
 - All the tools in one R package
 - Open source and customizable to fit production processes (input, reports, etc.)

Fit predefined Database schemas



- Databases store individual time series information per record, while one workspace is designed to contain multiple time series
 - One workspace per time-series is not practical
- Databases separate data, model specifications and external regressors in different tables or attributes
 - Workspaces store them all together



R package that helps handle all these aspects, based on
JSON specifications independent from data (JD_JSON)

- Specifications dynamically linked to the data via "series_name" field
 - Obtainable from workspaces
 - Convertible-back into workspaces
- } Interoperability with all the JD+ suite
- JD_JSON keys taken from **RJDemetra class c("SA_spec", "TRAMO_SEATS") attributes** + minor additions (ramps, intervention variables, external regr. handling, fixed coefficients)
 - Full and synthetic version (without key-values that are equal to default)

JD_JSON example



Series name

```
{  
  "series_name": "FATEXP 15",  
  "frequency": 12,  
  "method": "TS",  
  "spec": "RSA0",  
  "usrdef.outliersEnabled": true,  
  "usrdef.outliersType": ["AO", "LS"],  
  "usrdef.outliersDate": ["2009-09-01", "2009-03-01"],  
  "usrdef.varFromFile.infoList": [{  
    "container": "tdu02m.txt", "start": "2002-01-01", "n_var": 1,  
    "container": "lym_02.txt", "start": "2002-01-01", "n_var": 1  
  }],  
  "arima.mu": false,  
  "arima.p": 1,  
  "arima.d": 0,  
  "arima.q": 0,  
  "arima.bq": 0  
}
```

Default specification

```
  "usrdef.outliersEnabled": true,  
  "usrdef.outliersType": ["AO", "LS"],  
  "usrdef.outliersDate": ["2009-09-01", "2009-03-01"],  
  "usrdef.varFromFile.infoList": [{  
    "container": "tdu02m.txt", "start": "2002-01-01", "n_var": 1,  
    "container": "lym_02.txt", "start": "2002-01-01", "n_var": 1  
  }],  
  "arima.mu": false,  
  "arima.p": 1,  
  "arima.d": 0,  
  "arima.q": 0,  
  "arima.bq": 0  
}
```

External regressors (from files) information

JD_JSON.R

from_reduced_to_full_JD_JSON(...)
from_full_to_reduced_JD_JSON(...)

```
{  
  "series_name": "FATEXP_15",  
  "frequency": 12,  
  "method": "TS",  
  "spec": "RSA0",  
  "preliminary.check": true,  
  "estimate.from": "NA",  
  "estimate.to": "NA",  
  "estimate.first": "NA",  
  "estimate.last": "NA",  
  "estimate.exclFirst": 0,  
  "estimate.exclLast": 0,  
  "estimate.tol": 1e-07,  
  "estimate.eml": true,  
  "estimate.urfinal": 0.96,  
  "transform.function": "None",  
  "transform.fct": 0.95,  
  "usrdef.outliersEnabled": true,  
  ...  
  "arima.mu": false,  
  "arima.p": 1,  
  "arima.d": 0,  
  "arima.q": 0,  
  "arima.bp": 0,  
  "arima.bd": 1,  
  "arima.bq": 0,  
  "arima.coefEnabled": false,  
  "arima.coef": "NA",  
  "arima.coefType": "NA",  
  "fcst.horizon": -2,  
  "seats.predictionLength": -1,  
  "seats.approx": "Legacy",  
  "seats.trendBoundary": 0.5,  
  "seats.seasBoundary": 0.8,  
  "seats.seasBoundary1": 0.8,  
  "seats.seasTol": 2,  
  "seats.maBoundary": 0.95,  
  "seats.method": "Burman",  
  "ramps": "NA",  
  "intervention_variables": "NA",  
  "easterCoef": 0  
}
```

Basic DataBase schema: domain workspaces



Time series name	Metadata	Values	Domain
FATEXP_10	...	<10, 9.4, 8.9,...>	FAT
C_DEFL	...	<1.2, 3, 2.6,...>	FAT
...	...	...	FAT
VATPIA	...	<3.1, 3, 2.8,...>	TUR
VATAIA	...	<1.6, 2.2, 2.7,...>	TUR
...	...	...	TUR
...	...	...	Other domain

DATA table

Workspace (data+spec)	Domain
	FAT
	TUR
	Other domain

SPECIFICATIONS table

Need to be sure that a workspace contains all the series of its domain with updated data



DB schema with JD_JSON specifications (our choice)



Coherence ensured by design: no checks are needed

Time series name	Metadata	Values	Domain	Specifications (JD_JSON)
FATEXP_10	...	<10, 9.1, 8,...>	FAT	{...}
C_DEFL	...	<1.2, 3, 2.6,...>	FAT	{...}
...	...	...	FAT	{...}
VATPIA	...	<3.1, 3, 2.8,...>	TUR	{...}
VATAIA	...	<1.6, 2, 2.7,...>	TUR	{...}
...	...	...	TUR	{...}
...	...	...	Other domain	{...}

Basic Building Blocks – Object Oriented Architecture



JD_JSON_file_processor.R

- JD_JSON_file_processor (input_data_reader, ext_reg_data_reader, spec_file_name, output_workspace_dir, series_to_proc_names)

JD_JSON.R

- JD_JSON_from_materialized_workspace(...)
- JD_JSON_to_materialized_workspace(...)
- JD_JSON_to_virtual_workspace(...)
- JD_JSON_to_virtual_workspace (...)
- from_reduced_to_full_JD_JSON(...)
- from_full_to_reduced_JD_JSON(...)

Extended_tramoseats_spec.R - c("Extended_tramoseats_spec")

- Extends c("SA_spec", "TRAMOSEATS") from RJDemetra
- Constructor
 - to_tramoseats_spec_args(...)
- to_named_list(...)
 - from_named_list(...)
- to_sa_spec(...)
 - from_sa_spec(...)
- to_JD_JSON(...)
 - from_JD_JSON(...)

DATA_READERS

c ("Data_reader")

c ("Data_reader_ext_reg")

- Constructor • read_data(...) • read_data_ext_reg(...) • read_ext_reg_info(...)

Responsibility

Processing and output of the results

Interaction with workspaces and input-output of specifications

Interaction with RJDemetra

Acquisition of the input data (raw time series and ext.regressors)

Predefined interface methods

Workspace manager.R

- `get_single_ts_workspaces` (full_workspace, single_workspaces_path, compressed_ws=TRUE, ...)
- `merge_workspaces` (source_workspaces_path, merged_ws_name = "merged_ws", ...)
- `update_data` (workspace_xml_path, data_reader)
- `check_external_regressors_length` (workspace)
- `compare_workspaces` (workspace1, workspace2, output_file="differences_report.txt")

Responsibility

Operations with
workspaces
(split, merge, update
data, checks)

Reports.R

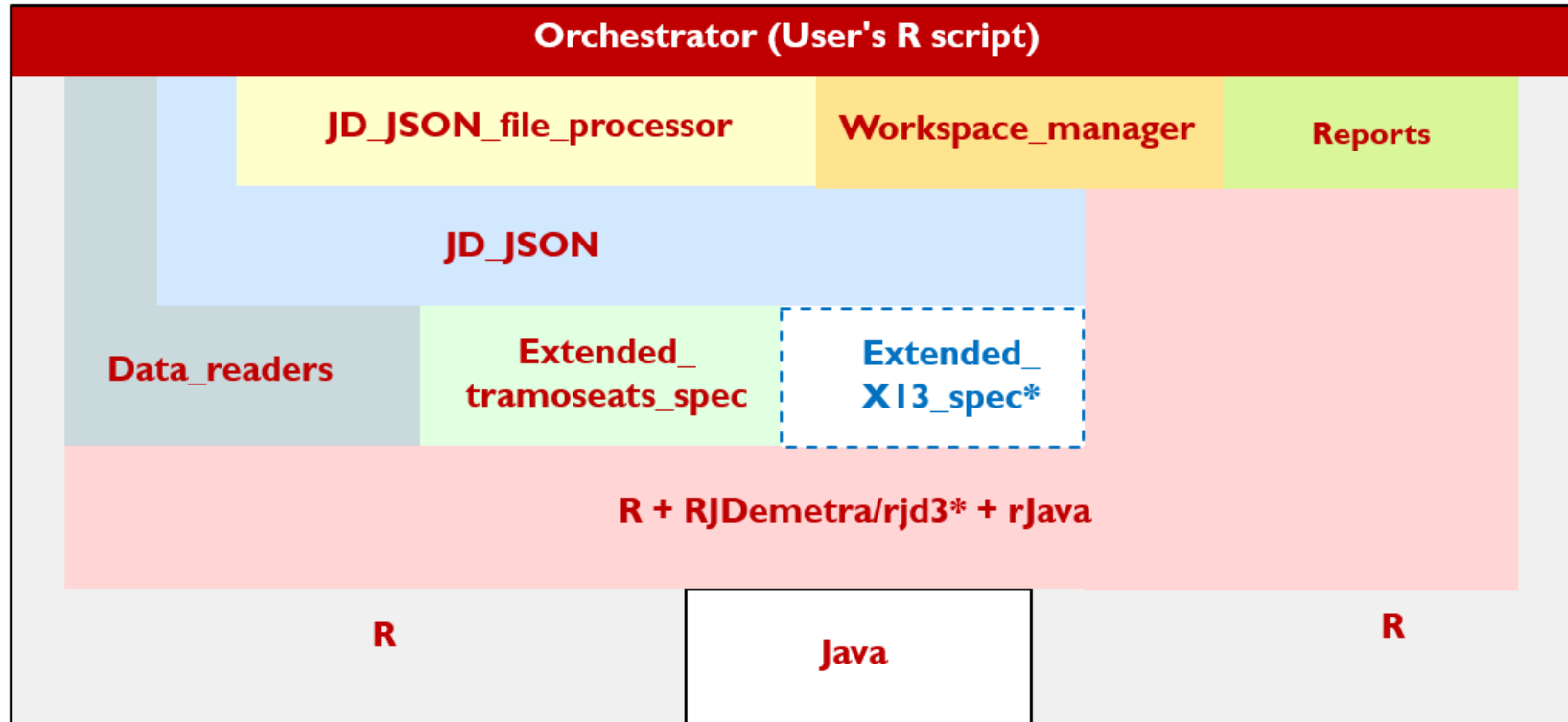
- `create_diagnostic_report1` (workspace, output_file="report.txt")
- `create_diagnostic_report2` (workspace, output_file="series_info.txt")
- `create_diagnostic_report_RJDQA` (workspace, output_file="RJDQA_report.pdf", ...)
- `compare_sa_ts` (new_model_workspace, old_model_workspace, ...)

Reporting,
revisions checks

Building blocks + work in progress(*)



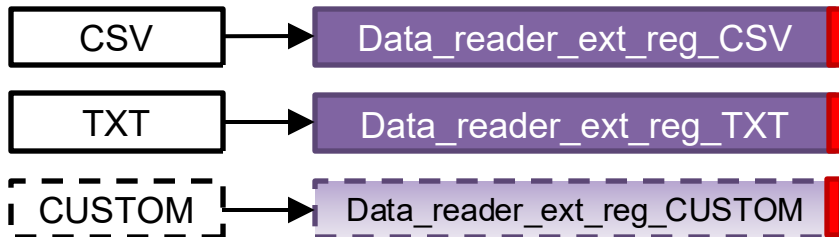
When a block lies on top of another, it means that it is built using it



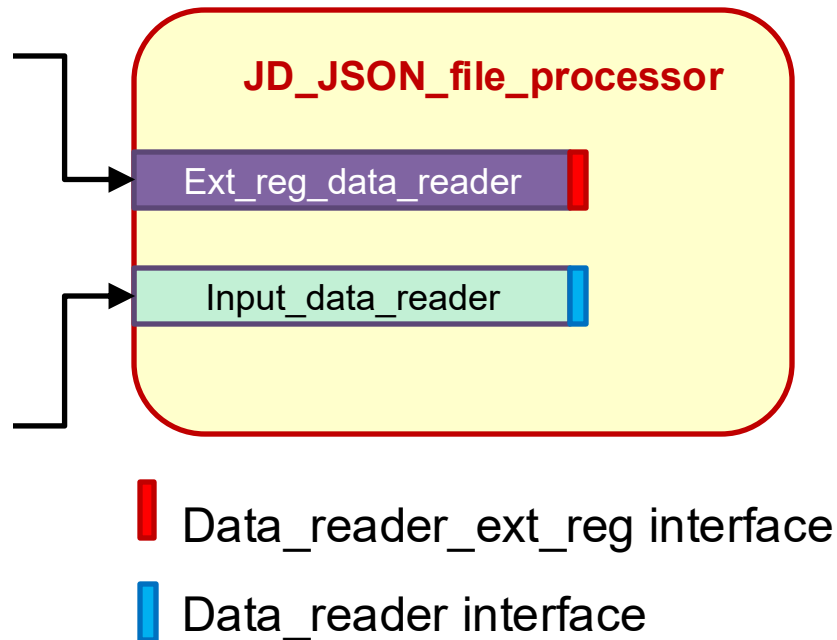
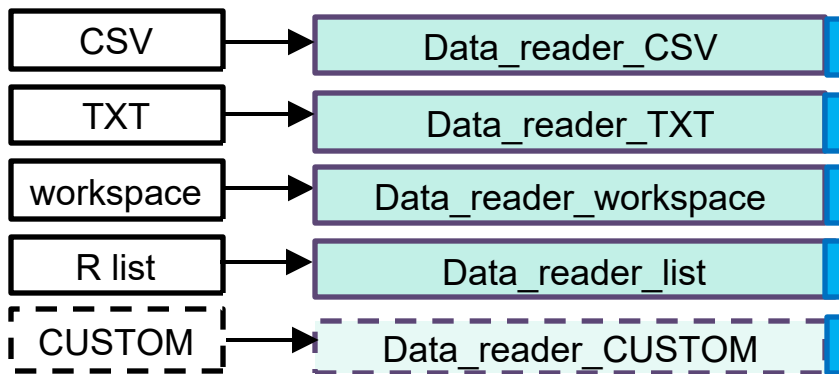
DataReaders: handling many input sources



Input_data_ext_reg

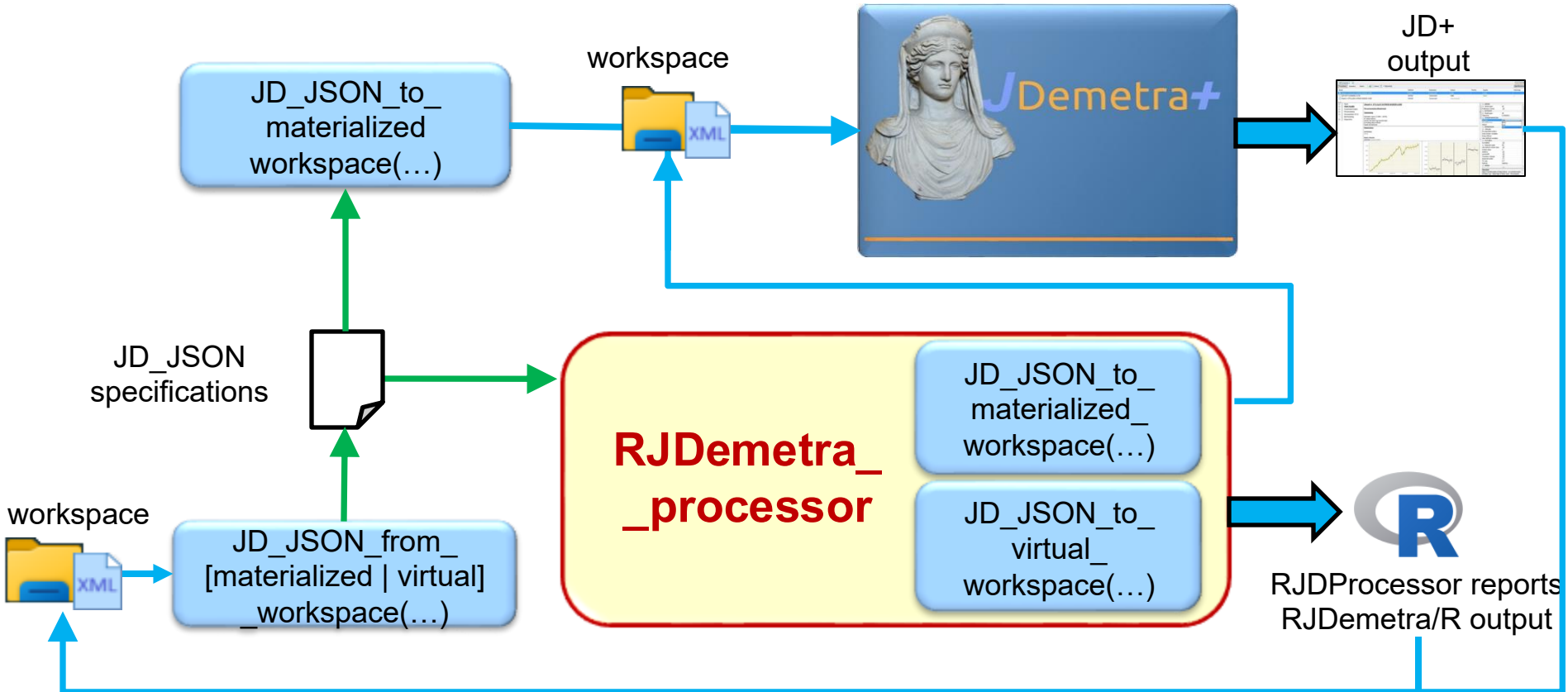


Input_data



Building a custom **DataReader** or **DataReaderExtReg** is possible by implementing it according to the **predefined interfaces**

Interoperability with JDemetra+ (GUI, Cruncher, ...) through workspace



Usage example (Orchestrator.R)



```
...
input_workspace_directory <- "C:\\Workspace-dir\\WS-FAT.xml" #Built with sa_ext plugin
input_data_file_name      <- "C:\\SITIC-FAT\\raw_data.csv"
regr_directory            <- "C:\\SITIC-FAT\\TS_regr" #Folder with external regressors
spec_file_name            <- "C:\\JD_JSON_specifications.txt"
diff <- TRUE               # Reduced JSON if diff=TRUE, Full JD_JSON format otherwise

##### Operational flow #####

input_data_reader          <- Data_reader_csv(input_data_file_name)
ext_reg_input_data_reader <- Provider_ext_reg_csv(regr_directory)

JD_JSON_from_materialized_workspace(input_workspace_directory,
                                     ext_reg_input_data_reader, JSON_file_name=spec_file_name, diff=diff)

virtual_ws <- JD_JSON_file_processor(input_data_reader, ext_reg_input_data_reader,
                                     spec_file_name, "output_workspace_container") #RJDemetra models

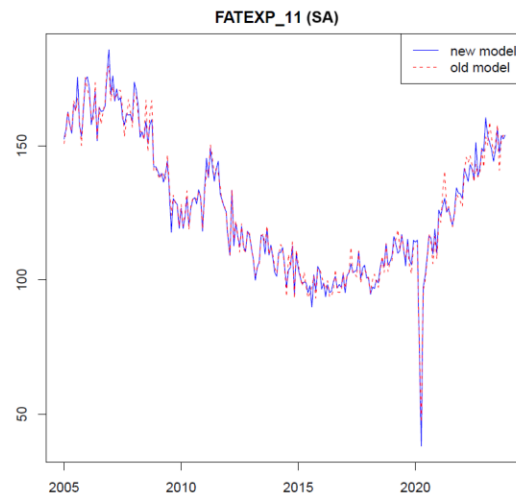
from_reduced_to_full_JD_JSON_file(spec_file_name, "JD_JSON_specifications_full.txt")
compare_sa_ts(new_model_workspace = virtual_ws, materialized_ws_new=FALSE
              old_model_workspace = input_workspace, materialized_ws_old=TRUE)

create_diagnostic_report1(virtual_ws, "report.txt")
```

Reports



TITLE: FATEXP_11				
BIC	LB_Prob	LB2_Prob	Norm_Prob	Norm_Test
859.19	0.748	0.000	0.112	4.39
Regressor	Coefficients	T-Stats	Not significant	
Week days	0.003	2.249		
Leap year	-0.012	-0.465	X	
AO (3-2020)	0.185	3.612		
AO (4-2020)	0.238	4.643		



PROBLEMS RESUME								
	Freq	TITLE	BIC	LB_Prob	LB2_Prob	Norm_Prob	Norm_Test	Not_significant_coeff
2	12	FATEXP_11	859.19	0.748	0 (*)	0.112	4.39	Leap year
3	12	FATEXP_13	751.01	0.919	0.031 (*)	0 (*)	154.77	
4	12	FATEXP_14	946.15	0.017 (*)	0.763	0.322	2.27	
5	12	FATEXP_15	958.01	0.021 (*)	0.07	0.339	2.16	
6	12	FATEXP_19	1036.34	0.711	0 (*)	0.748	0.58	AO (7-2020)
7	12	FATEXP_21	1020.76	0.883	0.079	0.036 (*)	6.66	
22	12	DIVID10	1107.37	0.04 (*)	0.291	0.021 (*)	7.73	Easter [6]
23	12	DIVIZ11	1458.61	0.674	0.848	0.002 (*)	12.19	

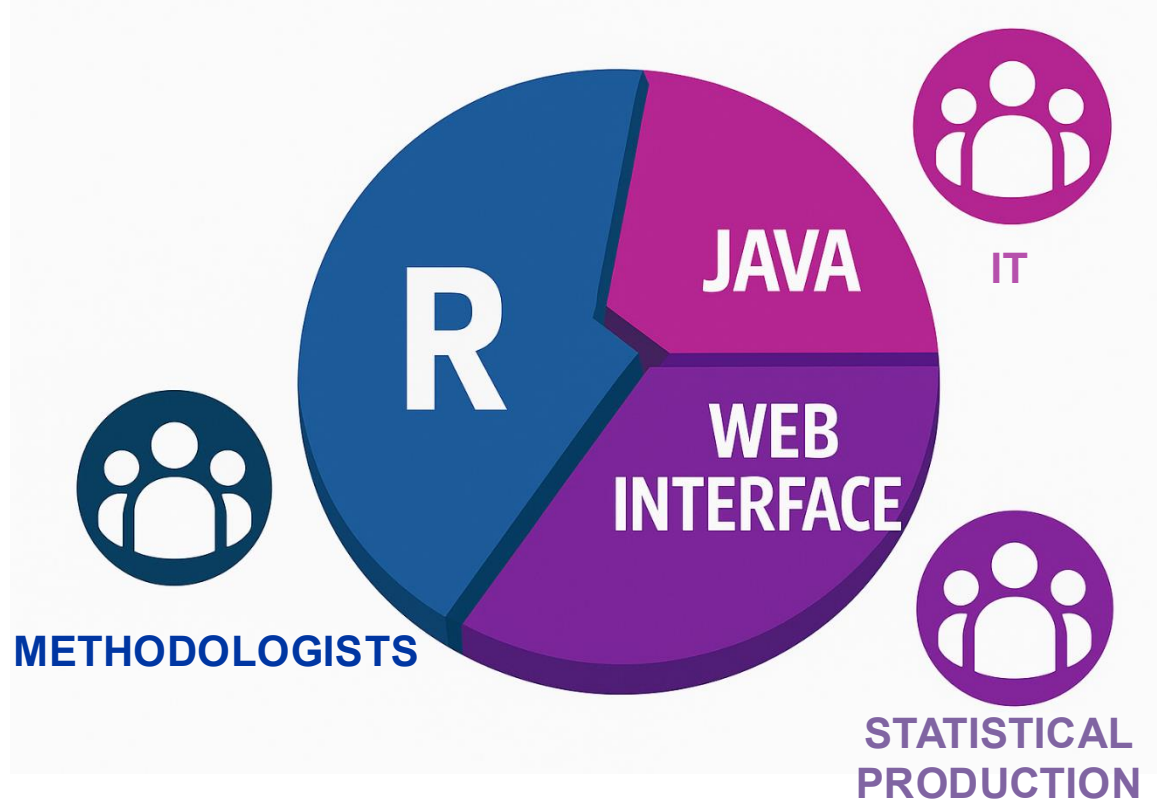
Different perspectives on the system



Every actor involved in seasonal adjustment works with different tools:

- suitable for its usage
- matching the user's skillset

Actors	Tool
Methodologists	RJDProcessor: produces and loads JSON specs
Statistical production staff	Web interface: loads data, shows specs and diagnostics
IT	Java module: reads JSON specs and computes seasonal adjustment



Web interface



S-Survey Management Application

sintesi.sviluppo.istat.it/#/pages/funzioni-statistiche/destagionalizzazione

SSurvey - Sistema Gestionale di Indagine

Light

Funzioni statistiche

Destagionalizzazione

Diffusione

Metadati

Strumenti

PREPARAZIONE DATI PER DESTAGIONALIZZAZIONE

IST - 01370 Indagine mensile su fatturato dell'industria

Pannello di controllo

Periodo di partenza della serie

01-2021

Ultimi 3 indicatori validati

Gennaio 2025

Dicembre 2024

Novembre 2024

NOTA : La base di riferimento utilizzata è BASE RIFERIMENTO 2021

AVVIA

Nuovo Chrome disponibile

Funzioni statistiche

Destagionalizzazione

Diffusione

Metadati

Strumenti

PREPARAZIONE DATI PER DESTAGIONALIZZAZIONE

IST - 01370 Indagine mensile su fatturato dell'industria

Storico destagionalizzazioni

Stato ↑↓	Data Esecuzione	Utente ↑↓	Periodo partenze (se specificato)	Modello	Time Series	Diagnostica	Errori	Elimina
INDIRETTA 70%	21/07/25 11:22:15							
100%	17/07/25 18:35:32							

Da 1 a 2 di 2 elementi totali

1

25

Prodotto sviluppato dalla Direzione Centrale per le Tecnologie Informatiche (DCIT) ISTAT 2023

Source code



Source code is available on GitHub:

<https://github.com/AlessandroPiovani/RJDProcessor>

https://github.com/AlessandroPiovani/JD_JSON_processor_Java

For information contact me at:

- alessandro.piovani@istat.it
- alessandro.piovani13@gmail.com

THANK YOU!

